

Notas de Aula #01: Operações com Bits e Bytes

1. BITS E BYTES:

1.1. Definição, Importância E Conversão

Quando falamos em computação, tudo se resume a informação digital representada por bits e bytes.

- **BIT** (*abreviação de binary digit*) é a menor unidade de informação em um sistema computacional e só pode assumir dois valores: 0 ou 1. Esses valores representam estados lógicos, como desligado/ligado ou falso/verdadeiro.
- **BYTE** é um agrupamento padrão de 8 bits. Essa organização facilita o armazenamento e o processamento de dados, permitindo representar desde simples caracteres de texto até números e instruções de programas.

A importância desses conceitos está em que todo o processamento, armazenamento e transmissão de dados em computadores e redes digitais é feito em termos de bits e bytes. Compreender essa base ajuda a entender como arquivos ocupam espaço, como a memória é medida (KB, MB, GB) e como a comunicação entre dispositivos é estruturada.

A relação entre bits e bytes é direta:

- Para converter de bytes para bits, basta multiplicar por 8.

Exemplo: 4 bytes = $4 \times 8 = 32$ bits.

- Para converter de bits para bytes, divide-se por 8.

Exemplo: 64 bits = $64 \div 8 = 8$ bytes.



Em Python, essa conversão pode ser expressa de forma simples com operações matemáticas, tornando fácil trabalhar com diferentes escalas de dados.

```
1 # -----
2 # Converter bytes para bits
3 def bytes2bits(bytes: int) → int:
4     return bytes * 8
5
6 # -----
7 # Converter bits para bytes
8 def bits2bytes(bits: int) → int:
9     return bits // 8
10
11 # -----
12 # Exemplos de uso
13 intBytes = 4
14 intBits = 64
15 print(f'{intBytes} bytes = {bytes2bits(intBytes)} bits')
16 print(f'{intBits} bits = {bits2bytes(intBits)} bytes')
```

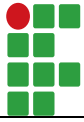
1.2. Conversão entre Binário, Decimal, Octal e Hexadecimal:

Os computadores trabalham internamente com base binária (0 e 1), mas diferentes bases numéricas são úteis em programação e eletrônica digital:

- **Decimal (base 10)**.....: sistema que usamos no dia a dia, com dígitos de 0 a 9.
- **Binário (base 2)**.....: formado apenas pelos dígitos 0 e 1; é a linguagem nativa dos computadores.
- **Octal (base 8)**.....: usa dígitos de 0 a 7; era muito usado em sistemas mais antigos e ainda aparece em permissões de arquivos em Unix/Linux.
- **Hexadecimal (base 16)**.....: utiliza dígitos de 0 a 9 e as letras A a F (que valem de 10 a 15). É bastante empregado para representar endereços de memória, cores em HTML/CSS e códigos de máquina.

Conversão Manual (ideia geral)

- **Decimal → Binário**.....: divide o número por 2 repetidamente, registrando os restos até chegar a 0, depois lê os restos de baixo para cima.
- **Decimal → Octal**.....: divide por 8, usando o mesmo processo.
- **Decimal → Hexadecimal** ...: divide por 16, convertendo os restos em 0–9 e A–F.



Conversões em Python

Python facilita a conversão entre bases com funções integradas:

```
1 # -----
2 intNum = 42 # número decimal de exemplo
3
4 # -----
5 # Decimal para outras bases
6 print(f'{intNum} em Binário = {bin(intNum)}') # Binário: '0b101010'
7 print(f'{intNum} em Octal = {oct(intNum)}') # Octal: '0o52'
8 print(f'{intNum} em Hexa = {hex(intNum)}') # Hex: '0x2a'
9
10 # -----
11 # De outras bases para decimal
12 print(int('101010', 2)) # Binário → Decimal: 42
13 print(int('52', 8)) # Octal → Decimal: 42
14 print(int('2a', 16)) # Hex → Decimal: 42
```

Observação:

- Os prefixos **0b**, **0o** e **0x** indicam, respectivamente, números binários, octais e hexadecimais.
- O segundo argumento de **int()** define a base de origem da *string* numérica.

Aplicações Práticas

- **Programação de baixo nível** : manipulação de endereços de memória e registradores.
- **Redes e segurança** : endereços IP, máscaras de sub-rede, criptografia.
- **Design e Web** : representação de cores (ex.: #FF5733 é hexadecimal).



1.3. Operadores Bit a Bit em Python

Em muitas aplicações de programação – como criptografia, compressão de dados, programação de sistemas embarcados e manipulação de flags – é necessário trabalhar diretamente com os bits que compõem um número. Para isso, o Python oferece operadores bit a bit (bitwise), que atuam sobre a representação binária dos inteiros.

Cada operador executa uma operação lógica em cada bit correspondente dos operandos.

Operador	Descrição	Exemplo	Resultado
&	AND: retorna 1 somente se ambos os bits forem 1.	6 & 3	2 (binário: 110 & 011 = 010)
	OR: retorna 1 se pelo menos um dos bits for 1.	6 3	6 (binário: 110 011 = 111)
^	XOR: retorna 1 se os bits forem diferentes.	6 ^ 3	5 (binário: 110 ^ 011 = 101)
~	NOT: inverte todos os bits (complemento de dois).	~6	-7 (inversão bit a bit de 6)
<<	Shift à esquerda: desloca os bits para a esquerda, preenchendo com 0. Equivale a multiplicar por 2 a cada deslocamento.	6 << 1	12 (binário: 110 → 1100)
>>	Shift à direita: desloca os bits para a direita, descartando os bits mais à direita. Equivale a dividir por 2 a cada deslocamento.	6 >> 1	3 (binário: 110 → 11)

```
1 # -----
2 intA = 6   # binário: 110
3 intB = 3   # binário: 011
4
5 # -----
6 print(f'\n{intA} em binário é {bin(intA)}')
7 print(f'\n{intB} em binário é {bin(intB)}')
8
9 # Operações bit a bit (bitwise)
10 print(f'\nOperações bit a bit entre {intA} e {intB}:\n')
11
12 print(f'\t{intA} & {intB} = {intA & intB}')      # AND → 2
13 print(f'\t{intA} | {intB} = {intA | intB}')      # OR  → 7
14 print(f'\t{intA} ^ {intB} = {intA ^ intB}')      # XOR → 5
15
16 print(f'\t~{intA} = {~intA}')                    # NOT  → -7
17 print(f'\t{intA} << 2 = {intA << 2}')            # Shift left 2 → 24
18 print(f'\t{intA} >> 1 = {intA >> 1}\n')
```

Nota sobre ~ (NOT):

Em Python, os inteiros são representados em complemento de dois para números negativos. Assim, ~x equivale a -(x + 1).



Aplicações Comuns

- **Manipulação de flags:** ativar, desativar ou verificar configurações específicas usando máscaras de bits.
- **Otimização de operações matemáticas:** por exemplo, usar $x \ll 1$ em vez de $x * 2$ em contextos de baixo nível.
- **Protocolos de comunicação e criptografia:** onde cada bit carrega significado próprio.

1.4. Manipulação de Bits e Máscaras

Manipulação de bits é a prática de trabalhar diretamente com os bits de um número para alterar, verificar ou extrair informações específicas.

Para facilitar essas operações, utiliza-se uma máscara de bits (bitmask): um valor binário em que apenas os bits de interesse estão definidos como 1.

Ao combinar a máscara com operadores bit a bit, podemos ligar (setar), desligar (limpar) ou verificar bits individuais.

Por que usar máscaras de bits?

- **Eficiência**.....: operar em nível de bits é extremamente rápido e economiza memória.
- **Controle fino**: ideal para representar múltiplas opções (flags) em um único número.
- **Aplicações práticas**: permissões de arquivos, configurações de hardware, protocolos de rede, compressão, criptografia.

Operações Comuns com Máscaras

Suponha um número de 8 bits representando 8 opções (de 0 a 7). Vamos trabalhar com a máscara para o bit 2 (terceiro bit a partir da direita): 0b00000100 (decimal 4).

Ação	Operação	Descrição
Ligar (setar) um bit	$\text{valor} = \text{mascara}$	Garante que o bit seja 1.
Desligar (limpar) um bit	$\text{valor} \&= \sim \text{mascara}$	Garante que o bit seja 0.
Alternar (toggle) um bit	$\text{valor} \wedge= \text{mascara}$	Inverte o estado do bit.
Verificar se um bit está ligado	$\text{valor} \& \text{mascara}$	Retorna diferente de 0 se o bit for 1.



```
1 # -----
2 bitMask = 0b00000100 # máscara para o bit 2
3 bitValor = 0b00000001 # apenas o bit 0 ligado
4
5
6 # -----
7 # Ligar bit 2
8 bitValor |= bitMask
9 print(bin(bitValor)) # 0b101
10
11 # Verificar bit 2
12 if bitValor & bitMask:
13     print('Bit 2 está ligado')
14
15 # Desligar bit 2
16 bitValor &= ~bitMask
17 print(bin(bitValor)) # 0b1
18
19 # Alternar bit 2 (inverter)
20 bitValor ^= bitMask
21 print(bin(bitValor)) # 0b101
```

Aplicações Reais

- **Permissões de Arquivos**: em sistemas Unix, permissões de leitura, escrita e execução são representadas por bits.
- **Dispositivos e Sensores.....**: cada bit de um registrador pode indicar o estado de uma porta ou sensor.
- **Protocolos de Rede**: cabeçalhos de pacotes usam bits individuais para sinalizar opções (ex.: flags em TCP/IP).

Esse tipo de manipulação é essencial quando se precisa de controle de baixo nível ou quando cada bit carrega uma informação independente dentro de um mesmo número.



1.5. Uso de `bit_length()`

O método `int.bit_length()` retorna o número mínimo de bits necessários para representar um número inteiro em binário, desconsiderando o sinal e os zeros à esquerda.

Ele é útil quando precisamos saber quantos bits são suficientes para armazenar ou transmitir um valor específico.

Por que é importante

- **Otimização de armazenamento**: ajuda a determinar o tamanho ideal de campos binários ou buffers.
- **Criptografia e compressão.....**: permite estimar a quantidade mínima de bits para representar chaves ou códigos.
- **Processamento de dados binários** : facilita cálculos que dependem da largura (número de bits) de um valor.

```
1 # -----
2 # Exemplo básico
3 print((0).bit_length())      # Saída: 0   (zero não precisa de bits)
4 print((5).bit_length())     # Saída: 3   (binário: 101)
5 print((255).bit_length())   # Saída: 8   (binário: 11111111)
6
7 # -----
8 # Uso em cálculos
9 intValor = 1023 # binário: 1111111111
10 print(intValor.bit_length()) # Saída: 10
11
12 # -----
13 # Determinar quantos bytes armazenar
14 bits = intValor.bit_length()
15 intBytesNeed = (bits + 7) // 8 # arredonda para cima
16 print(intBytesNeed)           # Saída: 2 bytes
```

Observação:

`bit_length()` ignora o sinal de números negativos. Por exemplo, `(-5).bit_length()` também retorna 3, pois a magnitude de 5 é representada com 3 bits.

Aplicações Comuns

- **Protocolos de comunicação**: definir campos de tamanho fixo suficiente para transmitir números.
- **Geração de chaves criptográficas** .: saber o número exato de bits em uma chave ou hash.
- **Armazenamento dinâmico**: calcular quantos bytes alocar para grandes inteiros.



1.6. Manipulação de Bytes em Python

Em Python, além de lidar com bits individuais, muitas aplicações exigem o trabalho direto com bytes — unidades de 8 bits que representam caracteres, dados binários ou sequências de informações.

Manipular bytes é essencial para processamento de arquivos binários, redes, criptografia, imagens, áudio e muito mais.

Tipos de Dados para Bytes

Python oferece dois tipos principais para representar dados em bytes:

- **bytes**
 - Imutável (como **str**);
 - Ideal para armazenar dados binários que não precisam ser alterados;
 - Exemplo: **b"Hello"** ou **bytes([72, 101, 108, 108, 111])**.
- **bytearray**
 - Mutável (permite alterações in-place);
 - Útil quando é necessário modificar o conteúdo;
 - Exemplo: **bytearray([72, 101, 108, 108, 111])**.

Criação e Acesso

```
1 # -----
2 # Criação a partir de string codificada em UTF-8
3 data = b"Python"          # bytes literal
4 print(data)               # b'Python'
5
6 # A partir de uma lista de inteiros (0 a 255)
7 data2 = bytes([80, 121, 116, 104, 111, 110])
8 print(data2)              # b'Python'
9
10 # Acesso individual retorna inteiros
11 print(data[0])            # 80 (valor ASCII de 'P')
12 print(chr(data[0]))       # 'P' (conversão para caractere)
```

Manipulação com bytearray

```
1 # -----
2 mutable = bytearray(b"Python")
3 mutable[0] = ord('J')     # altera o primeiro byte
4 print(mutable)            # bytearray(b'Jython')
```




Conversões Úteis

○ *String* ↔ *Bytes*

```
1 # -----  
2 texto = "Olá"  
3 b_texto = texto.encode("utf-8") # String para bytes  
4 print(b_texto)                  # b'Olá\x03\xa1'  
5 print(b_texto.decode("utf-8")) # Bytes para string
```

○ *Int* ↔ *Bytes*

```
1 # -----  
2 n = 1024  
3 b = n.to_bytes(2, byteorder="big") # Inteiro para bytes  
4 print(b)                           # b'\x04\x00'  
5 print(int.from_bytes(b, "big"))    # 1024 de volta para int
```

Operações Comuns

- Concatenação.....: `b1 + b2`
- Fatiamento: `data[1:4]`
- Iteração.....: `for byte in data: ...`

Aplicações Práticas

- Leitura/Escrita de Arquivos Binários..... : imagens, áudio, vídeos.
- Comunicação em Rede..... : protocolos TCP/UDP usam pacotes de bytes.
- Criptografia e Hashing..... : chaves e hashes são manipulados em forma de bytes.



1.7. `int.to_bytes()` e `int.from_bytes()` em Python

Em aplicações que envolvem arquivos binários, criptografia, redes ou qualquer forma de comunicação em baixo nível, é comum converter números inteiros em sequências de bytes e vice-versa.

O Python oferece dois métodos complementares para isso:

- `int.to_bytes()`: converte um inteiro em um objeto bytes.
- `int.from_bytes()`: reconstrói o inteiro original a partir de um objeto bytes.

Esses métodos são fundamentais para armazenar, transmitir ou interpretar números em formato binário.

`int.to_bytes()`

Sintaxe:

```
int.to_bytes(length, byteorder, *, signed=False)
```

Parâmetros:

- `length` : número de bytes que o resultado deve ter.
- `byteorder` : "**big**" ou "**little**", indicando a ordem dos bytes (**endianness**).
- `signed` (opcional) : **False** por padrão. Use **True** se precisar representar números negativos (com sinal em complemento de dois).

```
1 # -----
2 n = 1024
3 b = n.to_bytes(2, byteorder="big")
4 print(b) # b'\x04\x00'
5
6 # Versão little-endian
7 print(n.to_bytes(2, byteorder="little")) # b'\x00\x04'
```



`int.from_bytes()`

Sintaxe:

```
int.from_bytes(bytes_obj, byteorder, *, signed=False)
```

Parâmetros:

- `bytes_obj` : objeto bytes ou bytearray a ser convertido.
- `byteorder` : "**big**" ou "**little**", igual ao usado na gravação.
- `signed` (opcional) : **True** se os bytes representarem números negativos em complemento de dois.

```
1 # -----
2 b_big = b'\x04\x00'
3 b_little = b'\x00\x04'
4
5 print(int.from_bytes(b_big, "big"))      # 1024
6 print(int.from_bytes(b_little, "little")) # 1024
```

Aplicações Práticas

- **Protocolos de rede**.....: empacotar números em pacotes binários.
- **Criptografia e hashing**.....: manipular chaves e valores em formato binário.
- **Leitura/escrita de arquivos binários** ...: armazenar números em formatos compactos.

1.8. O que é `byteorder`

Quando usamos métodos como `int.to_bytes()` ou `int.from_bytes()`, precisamos especificar a ordem dos bytes em que o número será armazenado ou lido. Essa ordem é conhecida como **endianness**.

O parâmetro `byteorder` aceita duas opções principais:

- "**big**" : **Big-endian**; o byte mais significativo (MSB – most significant byte) vem primeiro (posição 0).
- "**little**" ...: **Little-endian**; o byte menos significativo (LSB – least significant byte) vem primeiro.



Diferença Prática

Considere o número inteiro **1024**.

Sua representação binária é **00000100 00000000** (dois bytes).

```
1 # -----
2 intValor = 1024
3
4 # Big-endian
5 print(intValor.to_bytes(2, byteorder="big"))    # b'\x04\x00'
6
7 # Little-endian
8 print(intValor.to_bytes(2, byteorder="little")) # b'\x00\x04'
```

Observe:

- Em big-endian, o byte mais significativo (\x04) aparece primeiro.
- Em little-endian, a ordem se inverte.

Revertendo de Bytes para Inteiro

Ao usar **from_bytes**, é crucial informar a mesma ordem utilizada na gravação para interpretar corretamente os valores.

```
1 # -----
2 binBig    = b'\x04\x00'
3 binLittle = b'\x00\x04'
4
5 print(int.from_bytes(binBig, "big"))    # 1024
6 print(int.from_bytes(binLittle, "little")) # 1024
```

Quando Usar Cada Ordem

- Big-endian
 - Padrão em muitos protocolos de rede (por exemplo, IP/TCP).
 - Mais legível para humanos, pois segue a ordem “mais significativo primeiro”.
- Little-endian
 - Padrão interno da maioria das arquiteturas x86 (PCs e servidores).
 - Pode ser mais eficiente em processadores com essa arquitetura.



Dica Rápida

Se você quiser saber a ordem de bytes nativa do seu sistema:

```
1 # -----  
2 import sys  
3  
4 print(sys.byteorder) # "little" ou "big"
```